

SBR-kompatibel PDF från HusHub med robust, underhållbar generering

Executive summary

Det mest hållbara sättet att få en PDF som "känns som SBR" utan att fastna i skör Word-layout eller dubbla renderingsmotorer är att göra **Alternativ B** till en tydlig standard: **SBR-kompatibelt innehåll + igenkännbar form**, men med en **förenklad, deterministisk print-layout** och **en enda kanonisk renderer** (Chromium via Playwright eller Puppeteer). Detta minskar risken för att PDF:n ändrar utseende beroende på driftläge, temp-utrymme eller fallback-logik. (pptr.dev) (playwright.dev)

Nyckelbeslut som bör tas explicit:

- **Kanonisk källa:** Word-mallen används som *specifikation* (struktur + kravtexter), men PDF genereras från **HTML/CSS** som är versionsstyrd och testad för print. Detta låter er hålla juridiska texter oförändrade men ändå ha robust layout. (pptr.dev)
- **Kanoniskt flöde:** Generera "officiell PDF-snapshot" **en gång när utlåtagendet läses/skickas**, lagra den och leverera samma fil vid varje nedladdning. Detta reducerar driftkostnad och eliminerar oförklarliga skillnader mellan preview och nedladdning. (Supabase Storage stödjer privata buckets + signed URLs för säker nedladdning.) (supabase.com) (supabase.com)
- **Driftsäker temp-hantering:** Puppeteer/Chromium skapar temporära profiler; om `/tmp` blir fullt kan ni få `ENOSPC ... mkdtemp '/tmp/puppeteer_dev_profile-XXXXXX'`. Det är ett känt produktionsmönster och ska lösas med dedikerad temp-yta (`TMPDIR`) och/eller `userDataDir` samt köad körning med concurrency-tak. (github.com) (nodejs.org) (pptr.dev)

Rekommendation: börja med **Playwright** (p.g.a. tydliga PDF-options och "persistent context" med `userDataDir`) men håll implementationen abstrakt så att ni kan byta motor. (playwright.dev) (playwright.dev)

Mappning från Word-mall till förenklad, SBR-kompatibel struktur

Strukturella block i Word-mallen som bör bli kanoniska HTML-sektioner

Utifrån er uppladdade mall (filnamn: "ÖB-K Utlåtande ... 2026.2") framgår en tydlig sektionering som lämpar sig väl för en förenklad men SBR-kompatibel PDF:

Omslag

- Titel ("Utlåtande över" / "Överlåtelsebesiktning").
- Objektidentifiering (minst fastighetsbeteckning och adress).
- Uppdragsnummer (UN) förekommer som ett fält i mallen (t.ex. "UN: Uppdragsnummer").

Rekommendation: Omslag ska vara en separat layouttyp ("cover") med egen typografisk skala och minimal sidinformation.

Innehållsförteckning

- Mallen har en explicit "INNEHÅLLSFÖRTECKNING" och tre bilagor.

Rekommendation: Implementera innehållsförteckningen som en deterministisk lista utan sidnummer (eller valbart sidnummer), för att undvika instabilitet vid små layoutändringar.

Huvuddel: Överlåtelsebesiktning för köpare

- Underrubriker: *Objekt, Uppdragsgivare, Besiktningsman*.
- Centrala metadatafält i tabeller:
- Objekt: fastighetsbeteckning, adress, kommun, fastighetsägare.
- Uppdragsgivare: namn, uppdragsnummer.
- Besiktningsman: namn, medlemsnummer, certifieringsnummer, telefon, e-post.
- Besiktningsuppdrag: omfattning, besiktningsdag/tid, närvarande.

Rekommendation: Dessa ska renderas med en stabil "label-value grid" (inte komplexa tabeller med radspanning), eftersom grid är lättare att göra deterministisk i print.

Handlingar och upplysningar

- "Tillhandahållna Handlingar" och "Upplysningar om fel i fastigheten".

Rekommendation: Rendera som tydliga block (rubrik + lista/paragraph). Håll det enkelt; undvik att blanda flera "kort" på samma sida om det ger oförutsägbara brytningar.

Okulär besiktning

- Mallen innehåller fält som "Särskilda förutsättningar ..." samt byggnadsdata (byggnadsår, om/tillbyggd, grundläggning, bjälklag, stomme, uppvärmning, fasad, fönster, yttertak, ventilation).

Rekommendation: Gör en "facts-sektion" där alla dessa fält finns, men tillåt tomma fält att döljas.

Noteringar

- Rubrik "NOTERINGAR".
- Underindelning: Huvudbyggnad → Utvändigt → Plan (källarplan/entréplan/övervåning/vind) + (valbart) Komplementbyggnad.
- För byggnadsdelar används mönster som "NOTERINGAR ... RISKANALYS ... FORTSATT TEKNISK UTREDNING" samt möjlighet att visa bilder.

Rekommendation: Standardisera detta till en enda HTML-komponent: **Observation** medfälten *notering, riskanalys, ftu, bilder och plats i byggnadshierarkin*.

Bilagor

- Bilaga 1: Villkor (version framgår i sidfoten som "Version 2026.1" i mallen).
- Bilaga 2: Liten byggordbok.
- Bilaga 3: Tekniska medellivslängder.

Rekommendation: Bilagor ska alltid starta på ny sida och vara "låsta" textblock i mallversionering.

Obligatorisk metadata (för SBR-känsla och spårbarhet)

Det här är minimum för att PDF ska upplevas som "officiell" och samtidigt vara spårbar:

- Objekt: fastighetsbeteckning, adress, kommun, fastighetsägare.
- Uppdrag: uppdragsnummer (UN), besiktningsdatum, klockslag (om ni använder), omfattning, närvarande.
- Utförare: besiktningsman (namn), medlemsnummer, certifieringsnummer, kontaktuppgifter.
- Dokumentadministration: mallversion (PDF-template), villkorsversion (Bilaga 1), snapshot-tidpunkt, dokument-ID.

SBR betonar generellt att en "av SBR godkänd besiktningsman" åtagit sig att följa SBR:s etiska regler och bestämmelser samt krav på ansvarsförsäkring; det stärker behovet av att era utlåtanden är konsekventa, spårbara och kan reproduceras. ¹

Juridiska texter: vad som måste hanteras som "låst"

Rekommendation (kritisk): Gör en tydlig gränsdragning i systemet mellan:

- *Data-drivet innehåll* (noteringar, riskanalys, tekniska observationer, bilder).
- *Låsta textblock* (Bilaga 1-villkor samt fasta standardformuleringar i huvuddelens uppdrags-/ansvarstexter).

Rent tekniskt bör låsta block vara versionsstyrda resurser (t.ex. `conditions/v2026.1.html`) och aldrig "omskrivas" av UI-texter.

Header/footer-element, logo och sidfotens "SBR-känsla"

Er Word-mall använder sidhuvud/sidfot som en tydlig identitetsbärare. I sidhuvud syns åtminstone etiketter för *Fastighetsbeteckning* och *Besiktningsdatum*. I sidfot syns (i mallen) en kombination av *Medlemsnummer*, devisen "Vår kunskap är din trygghet" samt version/copyright, plus sidnumrering ("sida ..."). På SBR:s webbplats syns devisen "Vår kunskap är din trygghet", vilket gör den till ett igenkännbart element ni bör efterlikna på ett kompatibelt sätt. ²

Rekommendation: Bevara *informationsinnehållet* i header/footer (fält och placering) men förenkla typografi och linjer så att det blir stabilt i print.

HTML/CSS-rekommendationer för print och stabil PDF-layout

Grundprincip: print-mall är en separat produkt

Puppeteer och Playwright genererar PDF med **print-media** som standard. Om previewn är "screen-layout" blir det nästan alltid mismatch. Båda motorerna dokumenterar att man kan emulera "screen" om man verkligen vill, men det ger ofta sämre printkvalitet och mer CSS-skörhet. (pptr.dev) (playwright.dev)

Rekommendation: Gör previewn till *print-preview* (samma CSS som PDF) istället för att försöka göra PDF "som skärmen".

CSS-pattern: @page + preferCSSPageSize + printBackground

- Styr papper och marginaler i CSS: `@page { size: A4; margin: ... }`. MDN beskriver `@page` som central i paged media. ³
- Aktivera i PDF-API: `preferCSSPageSize: true` så att `@page` prioriteras. Det stöds av Puppeteer och Playwright. ⁴
- Använd `printBackground: true` så att loggor/färgblock verkligen skrivs ut (default är annars ofta "nej"). ⁴
- Sätt `-webkit-print-color-adjust: exact` (och `print-color-adjust: exact`) eftersom både Puppeteer och Playwright påpekar att PDF annars får modifierade "print colors". ⁵

Exempel (CSS):

```
@media print {
  html { -webkit-print-color-adjust: exact; print-color-adjust: exact; }
}

@page {
  size: A4;
  margin: 18mm 16mm 18mm 16mm; /* top right bottom left */
}
```

Sidbrytningar: undvik "splittrade" noteringar och bilder

MDN listar paged media-egenskaper som `page-break-*`, `orphans`, `widows` och `@page`.³ Ni bör kombinera "moderna" `break-*` med bakåtkompatibla `page-break-*` för att få stabilt beteende i Chromium.

Exempel (CSS):

```
.page-break { break-before: page; page-break-before: always; }
.avoid-break { break-inside: avoid; page-break-inside: avoid; }
p { orphans: 3; widows: 3; }
```

`page-break-inside: avoid` / `break-inside: avoid` är ett klassiskt sätt att undvika sidbrytning i element.⁶

Fonts: gör dem lokala och vänta på laddning

Puppeteer har `waitForFonts` (default `true`) som väntar på `document.fonts.ready`. Det är viktigt för att undvika att PDF renderas innan webfonts ligger på plats, vilket annars kan ändra textmått och sidbrytningar.⁷

Rekommendation:

- Paketera era fonts i render-containern (eller som statiska assets med intern URL).
- Ha en explicit "preload/await fonts"-krok i renderjobbet för extra determinism.

Header/footer: templates i Chromium vs CSS margin boxes i paged-media motorer

Chromium-vägen (Playwright/Puppeteer): header/footer templates

Playwright dokumenterar att `page.pdf()` stödjer `displayHeaderFooter` och `headerTemplate/footerTemplate`, och noterar begränsningar: scripts körs inte och sidans styles är inte synliga i templates.⁸

Puppeteer har samma options i `PDFOptions` och specialklasser (`pageNumber`, `totalPages`, m.fl.).⁹

Exempel (footerTemplate):

```
<div style="font-size:9px; width:100%; padding:0 16mm; box-sizing:border-box;">
```

```

<div style="display:flex; justify-content:space-between; width:100%;">
  <div>Medlemsnummer: <span class="title"></span></div>
  <div>VÅR KUNSKAP ÄR DIN TRYGGHET • © ... • Version ...</div>
  <div>sida <span class="pageNumber"></span> / <span class="totalPages"></
span></div>
</div>
</div>

```

Paged-media-vägen (PrinceXML/WeasyPrint): CSS margin boxes

PrinceXML visar hur man kan sätta sidfot med `@page` och counters (`counter(page)` och `counter(pages)`), vilket ger trycksaks-lik kontroll. ¹⁰

Det här är en viktig skillnad i tradeoff-tabellen (se längre ner).

Rekommendation: Starta med header/footer templates i Playwright/Puppeteer (lägst implementationströskel i ert nuvarande stack), men designa mall-API:t så att ni senare kan byta till "margin boxes" om ni skulle välja Prince/WeasyPrint.

Minimal HTML-template (SBR-kompatibel, förenklad)

```

<!doctype html>
<html lang="sv">
<head>
  <meta charset="utf-8" />
  <title>Utlåtande - {{fastighetsbeteckning}} (UN {{uppdragsnummer}})</title>
  <style>
    html { -webkit-print-color-adjust: exact; print-color-adjust: exact; }
    body { margin: 0; font-family: Arial, sans-serif; font-size: 10.5pt;
line-height: 1.35; }
    @page { size: A4; margin: 18mm 16mm 18mm 16mm; }
    .page-break { break-before: page; page-break-before: always; }
    .avoid-break { break-inside: avoid; page-break-inside: avoid; }

    h1 { font-size: 20pt; margin: 0 0 8mm; }
    h2 { font-size: 14pt; margin: 10mm 0 4mm; }
    h3 { font-size: 11.5pt; margin: 6mm 0 2mm; }

    .cover { padding: 24mm 16mm; }
    .meta-grid { display: grid; grid-template-columns: 55mm 1fr; gap: 2mm
6mm; }
    .label { color: #444; }
    .value { font-weight: 600; }

    .kv { display: grid; grid-template-columns: 55mm 1fr; gap: 2mm 6mm; }
    .block { border: 1px solid #bbb; padding: 4mm; margin: 4mm 0; }
    figure { margin: 3mm 0 0; }
    figure img { max-width: 100%; height: auto; display: block; }
    figcaption { font-size: 9pt; color: #555; margin-top: 1mm; }
  </style>
</head>

```

```

<body>

<section class="cover avoid-break">
  <h1>Utlåtande över<br/>Överlåtelsebesiktning</h1>

  <div class="meta-grid">
    <div class="label">Fastighetsbeteckning:</div><div
class="value">{{fastighetsbeteckning}}</div>
    <div class="label">Adress:</div><div class="value">{{adress}}</div>
    <div class="label">Kommun:</div><div class="value">{{kommun}}</div>
    <div class="label">Fastighetsägare:</div><div
class="value">{{fastighetsagare}}</div>
    <div class="label">Uppdragsnummer (UN):</div><div
class="value">{{uppdragsnummer}}</div>
    <div class="label">Besiktningsdatum:</div><div
class="value">{{besiktningsdatum}}</div>
  </div>
</section>

<div class="page-break"></div>

<section class="avoid-break">
  <h2>Innehåll</h2>
  <ol>
    <li>Överlåtelsebesiktning för köpare</li>
    <li>Handlingar och upplysningar</li>
    <li>Okulär besiktning</li>
    <li>Noteringar</li>
    <li>Bilaga 1: Villkor</li>
    <li>Bilaga 2: Liten byggordbok</li>
    <li>Bilaga 3: Tekniska medellivslängder</li>
  </ol>
</section>

<section>
  <h2>Överlåtelsebesiktning för köpare</h2>
  <div class="block avoid-break">
    <div class="kv">
      <div class="label">Uppdragsgivare:</div><div>{{uppdragsgivare_namn}}
</div>
      <div class="label">Besiktningsman:</div><div>{{besiktningsman_namn}}
(medl.nr {{medlemsnummer}})</div>
      <div class="label">Certifiering:</div><div>{{certifieringsnummer}}</
div>
      <div class="label">Omfattning:</div><div>{{omfattning}}</div>
      <div class="label">Närvarande:</div><div>{{narvarande}}</div>
    </div>
  </div>
  <p>{{fast_text_uppdrag_ansvar}}</p>
</section>

```

```

<section>
  <h2>Handlingar och upplysningar</h2>
  <p><strong>Tillhandahållna handlingar:</strong> {{handlingar}}</p>
  <p><strong>Upplysningar om fel i fastigheten:</strong> {{upplysningar}}</
p>
</section>

<section>
  <h2>Okulär besiktning</h2>
  <p><strong>Särskilda förutsättningar:</strong>
  {{sarskilda_forutsattningar}}</p>
  <div class="block avoid-break">
    <div class="kv">
      <div class="label">Byggnadsår:</div><div>{{byggnadsar}}</div>
      <div class="label">Grundläggning:</div><div>{{grundlaggning}}</div>
      <div class="label">Stomme:</div><div>{{stomme}}</div>
      <div class="label">Ventilation:</div><div>{{ventilation}}</div>
    </div>
  </div>
</section>

<section>
  <h2>Noteringar</h2>
  {{noteringar_html}}
</section>

<div class="page-break"></div>
<section>
  <h2>Bilaga 1: Villkor</h2>
  {{bilaga1_html}}
</section>

<div class="page-break"></div>
<section>
  <h2>Bilaga 2: Liten byggordbok</h2>
  {{bilaga2_html}}
</section>

<div class="page-break"></div>
<section>
  <h2>Bilaga 3: Tekniska medellivslängder</h2>
  {{bilaga3_html}}
</section>

</body>
</html>

```

Visuell trohet: vad som ska bevaras vs vad som kan förenklas

Prioritet att bevara (hög "SBR-känsla", låg teknisk risk)

Struktur & rubrikhierarki

Den tydligaste igenkänningen kommer från *sektionordningen* och rubriknivåerna (t.ex. "Handlingar och upplysningar", "Okulär besiktning", "Noteringar", Bilaga 1–3), eftersom det är hur mottagaren orienterar sig. (Detta ligger i linje med SBR-positioneringen att man följer deras riktlinjer och bestämmelser som godkänd besiktningsman.) ¹

Header/footer-information

Bevara: fastighetsbeteckning, besiktningsdatum, medlemsnummer, sidnumrering. Det är i praktiken "dokumentets pass". Playwright/Puppeteer har robust stöd för sidnummer via template-klasser (`pageNumber`, `totalPages`). ¹¹

Juridiskt låsta formuleringar

Bevara ordalydelse i villkorstext och standardformuleringar. Detta är inte en layoutfråga utan en compliance- och riskfråga (spårbarhet + konsekvens). ¹

Bildhantering med caption

Bevara: bild + bildtext (caption) och referens till var i fastigheten bilden hör hemma. Rent CSS-mässigt är `figure/figcaption` stabilt och lätt att page-break-skydda. ⁶

Prioritet att förenkla (hög teknisk risk, låg "SBR-känsla" per investerad timme)

Pixelperfekt identisk Word-radbrytning

Små fontskillnader ger sidbryts-diffar. Försök inte matcha Word exakt. Lås istället er egen print-typografi.

Innehållsförteckning med sidnummer

Sidnummer-TOC blir skört vid varje liten ändring. Om ni vill ha navigering: använd PDF outline (Playwright stöder `outline`-option) eller länkar i webbversionen. ¹²

Avancerade "Word-formatting tricks" (spalter, inbäddade tabeller, flytande objekt)

Det är just sådant som tenderar att brytas i HTML→PDF och triggar regressions.

Teknisk arkitektur och driftkrav för pålitlig PDF-generering

Render-pipeline: en kanonisk renderer + snapshot-PDF

Både Puppeteer och Playwright beskriver att PDF genereras via `page.pdf()` och att output påverkas av print media, `printBackground`, `preferCSSPageSize` och färgjusteringar. ¹³

Rekommenderat produktbeteende:

- **Draft:** HTML-preview (samma print-CSS som PDF).
- **Lock/Send:** skapa snapshot av data + mallversion → rendera PDF en gång → lagra artefakten.
- **Download:** servera lagrad PDF (inte regenerera).

Supabase Storage stödjer både privata buckets och signed URLs (tidsbegränsade länkar) för säker delning, samt "download"-beteende. ¹⁴

Temp-utrymme, userDataDir/TMPDIR och varför ni får ENOSPC

Puppeteer skapar temporära Chromium-profiler; en känd incident är `ENOSPC: no space left on device, mkdtemp '/tmp/puppeteer_dev_profile-XXXXXX'` när `/tmp` fylls av `puppeteer_dev_*`-profiler över tid. ¹⁵

Node.js beskriver att `os.tmpdir()` på Unix kan styras av miljövariablerna `TMPDIR`, `TMP`, `TEMP` och annars defaultar till `/tmp`. ¹⁶

Puppeteer har `userDataDir` i `LaunchOptions`, dvs ni kan flytta profilen till en dedikerad arbetskatalog per jobb. ¹⁷

Playwright har `launchPersistentContext(userDataDir)` som explicit använder `userDataDir` för persistent storage. ¹⁸

Rekommendation (operativt minimum):

- sätt `TMPDIR=/work/tmp` i render-servicen
- sätt `userDataDir=/work/profile/<jobId>` per renderjobb
- städa `/work/<jobId>` efter lyckad render och efter timeout

Systemd-dokumentation varnar också för att `/tmp` och `/var/tmp` kan städas "ageing"-baserat (t.ex. 10 dagar för `/tmp`), vilket är ytterligare skäl att inte ha långlivade artefakter där. ¹⁹

Kö, concurrency-tak, cleanup, monitoring

Kö och concurrency-tak

Sätt ett lågt max (t.ex. 1–3 samtidiga Chromium-renders per worker-nod; exakt värde bestäms av CPU/memory). Syftet är att undvika spikar som både fyller temp och skapar flakig rendering.

Cleanup & timeouts

- hård job-timeout (t.ex. 60–120s)
- process cleanup (stäng browser/context även vid exception)
- radera jobbkataloger (profil + tmp + assets)
- larma om cleanup misslyckas (risk för ackumulerad tempanvändning)

Monitoring

Mät åtminstone: `pdf_job_duration`, `pdf_job_fail_rate`, `active_chromium_processes`, `tmp_disk_used`, `queue_depth`, `retry_count`.

Container-temp: tmpfs/emptyDir och vad som gäller

Om ni kör i containrar:

- Docker beskriver att `tmpfs` mount lagrar data i värdmaskinens minne, är ephemeral och lämpar sig för temporära filer/caching. ²⁰
- I praktiken mountar man ofta `tmpfs` till t.ex. `/tmp` eller `/work/tmp` (syntax varierar; officiella docs på vissa språk beskriver `--mount type=tmpfs,destination=...`). ²¹

- Kubernetes docs beskriver hur `emptyDir` kan användas och att `emptyDir.medium: Memory` ger tmpfs, samt att storlek kan påverkas av feature gates. ²²
- Kubernetes har även tydlig dokumentation om **ephemeral storage** requests/limits och att `emptyDir.sizeLimit` påverkar hur mycket som kan användas. ²³

Rekommendation: Om ni redan kört in i ENOSPC, börja med disk-backat `emptyDir` /volym med tydlig storlek (inte memory-backat) och flytta till tmpfs först när ni har bra monitorering.

Version pinning (kritisk)

Puppeteer dokumenterar att det fungerar bäst med den Chrome for Testing-version som Puppeteer laddar ner, och att det inte finns garanti med andra Chrome-versioner. ²⁴

Playwright varnar att det bara är garanterat kompatibelt med den bundlade browsern och att andra versioner sker "use at your own risk". ²⁵

Rekommendation: Pinna:

- Playwright/Puppeteer version
- Chromium/Chrome for Testing version (implicit via biblioteket)
- PDF-template version (er)
- Villkorstext version (Bilaga 1)

Mermaid: rendering pipeline (flowchart)

```
flowchart TD
    A[Draft i HusHub] --> B[Render HTML med print-CSS]
    B --> C[Preview i webben]
    A --> D[Lås/Skicka utlåtande]
    D --> E[Skapa data-snapshot + mallversion]
    E --> F[Enqueue PDF-jobb]
    F --> G[PDF-worker: Chromium (Playwright/Puppeteer)]
    G --> H[Job workspace: TMPDIR + userDataDir per job]
    H --> I[page.pdf: preferCSSPageSize + printBackground]
    I --> J[Lagra PDF-snapshot i Storage]
    J --> K[Skicka e-post med webblänk + PDF-länk]
    K --> L[Nedladdning: servera samma PDF]
    G --> M[Cleanup + metrics]
```

Mermaid: deployment architecture (containers, volumes, tmp handling)

```
flowchart LR
    subgraph APP[HusHub App]
        API[API/UI]
        DB[(DB: reports, snapshots)]
        API --> DB
        API --> Q[Queue]
    end

    subgraph RENDER[PDF Render Service]
        W[Workers]
    end
```

```

VOL[(Work volume /work)]
TMP[(TMPDIR /work/tmp)]
PROF[(Profiles /work/profile)]
W --- VOL
W --- TMP
W --- PROF
end

subgraph STORAGE[Storage]
  S[(PDF bucket: private)]
end

Q --> W
W --> S
API --> S

```

Migrering från dual-engine till single canonical renderer, fallback-policy och QA

Migreringsstrategi (beslutsorienterad)

Steg 1: Definiera "kanonisk PDF-HTML"

Skapa en renderfunktion som tar *data snapshot + template version* och producerar HTML enligt den förenklade strukturen.

Steg 2: Gör preview till samma HTML/CSS

Undvik två olika presentationslager (React-view vs PDF-view). Preview ska renderas som "print preview" så att skillnader fångas tidigt. Att Puppeteer/Playwright renderar i print-media som default gör detta extra viktigt. ⁵

Steg 3: Byt "Ladda ner PDF" till snapshot-PDF

Generera PDF vid "Läs/Skicka" och lagra den. Servera samma fil därefter (se Supabase-delen). ²⁶

Steg 4: Gör structured PDF till explicit fallback (inte tyst)

Fallback får inte se ut som "fel PDF". Policy:

- Om kanonisk render failar → retry (snabb backoff)
- Om fortfarande fail → markera PDF-status = failed och visa tydligt fel + "försök igen", eller leverera en "Förenklad PDF" med watermark och loggad incident.

QA/test-checklista och acceptanskriterier

Visuella regressions

- Golden-PDF per scenario (liten/normal/stor rapport, många bilder, bilagor, komplementbyggnad).
- Rasterisera PDF-sidor till bilder och kör pixel-diff med tolerans.
- Kontrollera "kritiska brytpunkter": varje bilaga startar ny sida; notering + bild bryts inte mitt i block. (CSS `break-inside/page-break-inside: avoid`.) ⁶

Font rendering

- Verifiera att samma fonts används i body och att fonts laddas innan PDF (Puppeteer `waitForFonts`).

7

Header/footer correctness

- Sidnumrering korrekt på alla sidor.
- Fält för fastighetsbeteckning/besiktningsdatum/medlemsnummer presenteras.
- Om ni använder header/footer templates: verifiera att templates har inline-CSS (Playwright varnar att page styles inte syns i templates). 11

Reproducerbarhet & snapshots

- Samma snapshot ska alltid ge samma PDF (hash på input + template version).
- Äldre länkar ska alltid peka på samma PDF-artefakt (immutability).

Tillgänglighet (om krav finns)

- Playwright har `tagged` (accessible PDF) som option. Sätt explicit om krav finns och testa med PDF-verktyg. 8

Minimal sample job script (Playwright)

```
import fs from 'node:fs/promises'
import path from 'node:path'
import os from 'node:os'
import { chromium } from 'playwright'

function tmpRoot() {
  return process.env.TMPDIR || os.tmpdir()
}

async function mkJobDir(jobId) {
  const dir = await fs.mkdtemp(path.join(tmpRoot(), `hushub-pdf-${jobId}-`))
  const profileDir = path.join(dir, 'profile')
  await fs.mkdir(profileDir, { recursive: true })
  return { dir, profileDir }
}

export async function renderPdf({ jobId, html, outputPath, headerTemplate,
  footerTemplate }) {
  const { dir, profileDir } = await mkJobDir(jobId)

  const context = await chromium.launchPersistentContext(profileDir, {
    headless: true })
  try {
    const page = await context.newPage()
    await page.setContent(html, { waitUntil: 'domcontentloaded' })

    // Extra determinism: vänta på fonts + bilder
    await page.evaluate(async () => {
      if (document.fonts?.ready) await document.fonts.ready
      await Promise.all(Array.from(document.images).map(img =>
```

```

img.complete ? true : new Promise(res => {
  img.addEventListener('load', res, { once: true })
  img.addEventListener('error', res, { once: true })
  })))
})

await page.pdf({
  path: outPath,
  printBackground: true,
  preferCSSPageSize: true,
  displayHeaderFooter: true,
  headerTemplate,
  footerTemplate,
  // marginaler måste vara tillräckliga för header/footer
  margin: { top: '22mm', bottom: '18mm', left: '16mm', right: '16mm' },
  tagged: true,
})
} finally {
  await context.close()
  await fs.rm(dir, { recursive: true, force: true })
}
}

```

Playwright dokumenterar `page.pdf()` (print media, preferCSSPageSize, printBackground, tagged) och `launchPersistentContext(userDataDir)` för att kontrollera profildata. ²⁷

Minimal sample job script (Puppeteer)

```

import fs from 'node:fs/promises'
import path from 'node:path'
import os from 'node:os'
import puppeteer from 'puppeteer'

function tmpRoot() {
  return process.env.TMPDIR || os.tmpdir()
}

export async function renderPdf({ jobId, html, outPath, headerTemplate,
  footerTemplate }) {
  const base = await fs.mkdtemp(path.join(tmpRoot(), `hushub-pdf-${jobId}-`))
  const userDataDir = path.join(base, 'profile')
  await fs.mkdir(userDataDir, { recursive: true })

  const browser = await puppeteer.launch({ headless: true, userDataDir })
  try {
    const page = await browser.newPage()
    await page.setContent(html, { waitUntil: 'domcontentloaded' })
    await page.pdf({
      path: outPath,
      printBackground: true,

```

```

    preferCSSPageSize: true,
    displayHeaderFooter: true,
    headerTemplate,
    footerTemplate,
  })
} finally {
  await browser.close()
  await fs.rm(base, { recursive: true, force: true })
}
}

```

Puppeteer dokumenterar `Page.pdf()` (print media och `emulateMediaType('screen')` vid behov), samt `PDFOptions` (printBackground, preferCSSPageSize, waitForFonts) och `LaunchOptions` `userDataDir`. ²⁸

Val av motor: tradeoffs och rekommenderade use cases

Alternativ	Fidelity (webb→PDF)	JS-support	Paged-media features	Kostnad	Operativ komplexitet	F
Puppeteer (Chromium)	Hög, särskilt om ni redan har webblayouten ²⁹	Full (Chromium) ³⁰	Bra grund via <code>@page</code> + <code>PDFOptions</code> + templates ³¹	OSS	Medel-hög (browser lifecycle + <code>/tmp /</code> profile + pinning) ³²	M
Playwright (Chromium)	Hög, liknande Puppeteer ⁸	Full (Chromium) ⁸	Bra PDF-API (preferCSSPageSize, printBackground, tagged) ⁸	OSS	Medel-hög men tydlig modell med bundlad browser + persistent context ³³	F
wkhtmltopdf	Medel; bygger på Qt WebKit (äldre motor) ³⁴	Begränsat jämfört med modern Chromium	Bra "klassisk" HTML→PDF men svagare modern CSS	OSS (LGPLv3) ³⁴	Medel (binär drift, men mindre browser-komplexitet)	M
PrinceXML (vendor: YesLogic ³⁵)	Mycket hög för trycksaks-layout (paged media) ¹⁰	Begränsat/ print-fokuserat	Mycket starkt: margin boxes, counters, footnotes, m.m. ¹⁰	Kommersiell; site license start ca US\$2000/år ³⁶	Medel (licens + binär), ofta stabilare än Chromium för print	M
WeasyPrint	Medel-hög för statiska dokument; mindre "browser quirks"	Ingen JS; "no JavaScript, no live rendering" ³⁷	Starkt fokus på paginering (men inte full browser-CSS)	OSS; support finns kommersiellt ³⁸	Medel (Python dependencies)	M

Praktisk slutsats för HusHub "Alternativ B": börja med Playwright/Puppeteer för att matcha er webbaserade modell och lås ner drift+templating. Utvärdera PrinceXML bara om ni senare behöver trycksaksfunktioner som margin boxes/counters på ett sätt Chromium inte ger er stabilt. ³⁹

Nyckeldokument (primära källor)

- Puppeteer `Page.pdf()` och `print/screen media + print color adjust` ³⁰
- Puppeteer `PDFOptions` (`preferCSSPageSize`, `printBackground`, `waitForFonts`) ⁷
- Puppeteer `LaunchOptions.userDataDir` ¹⁷
- Playwright `page.pdf()` (`preferCSSPageSize`, `printBackground`, `tagged`, `print media`) ⁸
- Playwright `launchPersistentContext(userDataDir)` ¹⁸
- MDN Paged Media / page-break-inside ⁴⁰
- Systemd: `/tmp ageing/cleanup` ¹⁹
- Node.js: `os.tmpdir()` och `TMPDIR / /tmp` ¹⁶
- Supabase Storage: private buckets + signed URLs + downloads ¹⁴
- SBR: roll/krav för "av SBR godkänd besiktningsman" och ansvar/regler ¹
- SBR: devis och organisationsbeskrivning ²

Frågor att klargöra innan implementation

- 1) Är "officiell PDF" en **immutabel snapshot** efter "Skicka", eller kan den uppdateras i efterhand? (Rekommendation: immutabel + ny version vid ändring.)
- 2) Vilken är er **minsta accepterade SBR-känsla**: header/footer-layout och rubrikstruktur, eller krävs att omslaget saknar header/footer?
- 3) Ska Bilaga 1–3 alltid ingå, eller ska vissa bilagor kunna exkluderas per uppdragstyp?
- 4) Bildpolicy: max antal bilder, komprimering, och krävs bildtext/position alltid?
- 5) Är PDF:n tänkt att bifogas i mäklaravtal med strikta krav på sidnumrering/format?
- 6) Driftmiljö: kör ni renderingen i VM, i Docker ⁴¹, eller i Kubernetes-kuster? (Påverkar temp/volym-design.) ⁴²
- 7) Tillgänglighetskrav: ska PDF vara "taggad" (accessible)? (Playwright/Puppeteer har stöd men kräver QA.) ¹²

Instruktion till Codex

Du är en engineering assistant. Implementationsdetaljer är öppna, men lösningen **måste** uppfylla beteende, kontrakt och acceptanskriterier nedan.

Målbeteende

- Systemet ska producera en **SBR-kompatibel PDF-snapshot** (Alternativ B) som:
 - följer Word-mallens sektionordning och rubrikhierarki
 - inkluderar låsta juridiska texter (Bilaga 1 + standardblock) exakt enligt versionsstyrda resurser
 - har stabil header/footer med fastighetsbeteckning, besiktningsdatum, medlemsnummer och sidnumrering
- PDF ska genereras **en gång** vid statusövergång `draft → locked` eller `draft → sent` (konfigurerbart), lagras och återanvändas vid nedladdning.
- Webb-preview ska använda **samma HTML/CSS** som PDF (print-preview), för att eliminera mismatch.

Inputs

- `report_id`
- `snapshot_id` (immutable data snapshot)
- `template_version` (ex: `pdf_template_v1`)
- `conditions_version` (ex: `sbr_conditions_2026_1`)

- Dataobjekt:
- objektdata (fastighetsbeteckning, adress, kommun, fastighetsägare)
- uppdragsdata (UN, besiktningsdatum/tid, omfattning, närvarande)
- besiktningsman (namn, medlemsnummer, certifieringsnummer, kontakt)
- handlingar/upplysningar
- okulär byggnadsdata
- noteringar (hierarki: byggnad→del→rum/plan; fält: notering, risk, ftu; bilder med caption)

Outputs

- En PDF-fil (bytes) + metadata
- Lagrad fil i Supabase ⁴³ Storage (private bucket) med kontrollerad åtkomst via signed URL eller auth-download. ²⁶

Renderingkrav (oavsett motor)

- Använd Chromium-baserad PDF (Playwright eller Puppeteer).
- PDF options måste inkludera:
- `preferCSSPageSize: true`
- `printBackground: true`
- header/footer påslaget via templates (eller motsvarande) med sidnummer
- CSS måste inkludera:
- `@page` A4 + marginaler
- `break-inside: avoid` / `page-break-inside: avoid` för block som inte får splittras
- `-webkit-print-color-adjust: exact`
- Fonts/logos/images ska vara lokalt åtkomliga och får inte kräva extern internetåtkomst.

Drift och felhantering

- Per jobb:
- skapa job workspace-katalog
- sätt `TMPDIR` till workspace tmp
- sätt `userDataDir` (eller persistent context dir) till workspace profile
- städa workspace vid success och failure
- Concurrency:
- globalt tak per worker (konfigurerbart)
- köad körning med retry på transient fel
- Felpolicy:
- `transient` (timeout, ENOSPC, chromium launch) → retry N gånger
- `permanent` (template validation error) → fail utan retry
- Vid fail: markera PDF-status och exponera begripligt fel i UI; leverera inte "tyst" annan layout.

Lagringsschema (fält att spara)

Spara minst följande i DB (tabell t.ex. `report_pdf_snapshots`):

- `id`
- `report_id`
- `snapshot_id`
- `template_version`
- `conditions_version`
- `pdf_storage_bucket`

- `pdf_storage_path`
- `pdf_sha256`
- `pdf_generated_at`
- `status` (`pending` | `generated` | `failed`)
- `error_code` (nullable)
- `error_message` (nullable)
- `render_engine` (`playwright` | `puppeteer`)
- `chromium_version` (om tillgängligt)
- `metrics` (JSON: `duration_ms`, `retries_count`, `pages_count`)

Acceptanskriterier

- **Determinism:** Samma `snapshot_id + template_version + conditions_version` ger samma `pdf_sha256`.
- **Ingen tyst fallback:** Om kanonisk render misslyckas får användaren inte automatiskt en annan layout utan tydlig markering.
- **Visuell stabilitet:** Golden tests (rasterized PDF diff) passerar för minst 5 representativa rapporttyper.
- **Header/footer:** Sidnumrering och metadata syns på alla sidor enligt mallens krav.
- **Bilagor:** Bilaga 1–3 startar alltid på ny sida.
- **Drift:** Inga ENOSPC-fel vid belastningstest inom konfigurerad concurrency och diskbudget; `TMPDIR/userDataDir` är aldrig `/tmp` i produktion.

1 <https://sbr.se/sok-besiktningsman/overlatelsebesiktning/vad-innebar-av-sbr-godkand-besiktningsman/>

<https://sbr.se/sok-besiktningsman/overlatelsebesiktning/vad-innebar-av-sbr-godkand-besiktningsman/>

2 <https://sbr.se/>

<https://sbr.se/>

3 40 https://mdn2.netlify.app/en-US/docs/Web/CSS/Paged_Media

https://mdn2.netlify.app/en-US/docs/Web/CSS/Paged_Media

4 7 31 <https://pptr.dev/api/puppeteer.pdfoptions>

<https://pptr.dev/api/puppeteer.pdfoptions>

5 13 28 29 30 43 <https://pptr.dev/next/api/puppeteer.page.pdf>

<https://pptr.dev/next/api/puppeteer.page.pdf>

6 <https://developer.mozilla.org/uk/docs/Web/CSS/page-break-inside>

<https://developer.mozilla.org/uk/docs/Web/CSS/page-break-inside>

8 11 12 27 39 <https://playwright.dev/docs/api/class-page>

<https://playwright.dev/docs/api/class-page>

9 PDFOptions interface | Puppeteer

https://pptr.dev/api/puppeteer.pdfoptions?utm_source=chatgpt.com

10 <https://www.princexml.com/doc/paged/>

<https://www.princexml.com/doc/paged/>

14 26 <https://supabase.com/docs/guides/storage/buckets/fundamentals>

<https://supabase.com/docs/guides/storage/buckets/fundamentals>

15 32 Better way to cleanup puppeteer_dev_chrome_profile-* · Issue #6414 · puppeteer/puppeteer · GitHub

https://github.com/puppeteer/puppeteer/issues/6414?utm_source=chatgpt.com

16 <https://nodejs.org/api/os.html>

<https://nodejs.org/api/os.html>

17 <https://pptr.dev/api/puppeteer.launchoptions>

<https://pptr.dev/api/puppeteer.launchoptions>

18 33 <https://playwright.dev/docs/api/class-browsertype>

<https://playwright.dev/docs/api/class-browsertype>

19 https://systemd.io/TEMPORARY_DIRECTORIES/

https://systemd.io/TEMPORARY_DIRECTORIES/

20 42 <https://docs.docker.com/engine/storage/>

<https://docs.docker.com/engine/storage/>

21 <https://docs.docker.top/engine/storage/tmpfs/>

<https://docs.docker.top/engine/storage/tmpfs/>

22 41 <https://kubernetes.io/es/docs/concepts/storage/volumes/>

<https://kubernetes.io/es/docs/concepts/storage/volumes/>

23 <https://kubernetes.io/docs/concepts/storage/ephemeral-storage/>

<https://kubernetes.io/docs/concepts/storage/ephemeral-storage/>

24 <https://pptr.dev/api/puppeteer.puppeteernode.launch>

<https://pptr.dev/api/puppeteer.puppeteernode.launch>

25 <https://playwright.bootcss.com/docs/1.3.0/api/class-browsertype>

<https://playwright.bootcss.com/docs/1.3.0/api/class-browsertype>

34 <https://wkhtmltopdf.org/index>

<https://wkhtmltopdf.org/index>

35 36 https://www.princexml.com/purchase/license_faq/

https://www.princexml.com/purchase/license_faq/

37 <https://manpages.ubuntu.com/manpages/jammy/man1/weasyprint.1.html>

<https://manpages.ubuntu.com/manpages/jammy/man1/weasyprint.1.html>

38 <https://weasyprint.org/index.html>

<https://weasyprint.org/index.html>